

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA

MASTER DEGREE IN COMPUTER ENGINEERING

Course of **Software Process Engineering**

The Impostor

Arcese Gabriele

matr: 0001183007

Coli Diego

matr: 0001172691

Costantini Marco

matr: 0001189584

Meco Daniel

matr: 0001192138

Academic Year 2025/2026

May 13, 2026

Contents

1	Analysis	3
1.1	Problem analysis	3
1.1.1	Context	3
1.1.2	Problem statement	3
1.1.3	Proposed solution	3
1.1.4	Roles	3
1.2	Requirements analysis	3
1.2.1	System requirements	3
1.2.2	Functional requirements	4
1.2.3	Non-functional requirements	5
1.3	Domain model	5
1.3.1	Ubiquitous language	5
1.3.2	Bounded context	6
2	Design	7
2.1	Engine Context	7
2.2	Lobby Context	9
2.3	Comm Context	10
2.4	Mockups	11
3	Architecture	20
3.1	Physical architecture	20
3.2	Logical architecture	21
4	Implementation	22
4.1	Technologies used	22
4.2	Game Engine implementation (Go)	22
4.2.1	The Aggregate Root and the Event-Driven approach	22
4.2.2	Thread-safety in volatile memory	23
4.2.3	Asynchronous execution and Goroutines	24
4.3	Client implementation (Vue.js)	25
4.3.1	State management (Pinia)	25
4.3.2	Real-time communication (WebSocket)	26
4.3.3	Voice chat (WebRTC)	27
4.3.4	Backend integration (REST API)	29
4.4	Gateway implementation	30
4.4.1	Real-time lobby synchronization	30
4.4.2	In-game text chat and WebRTC signaling relay	30
4.4.3	Game Engine bridge	31
4.5	Lobby Service implementation	32
4.5.1	Architectural design	32
4.5.2	Room management and security	33
4.5.3	Integration with the Comm Service	34

5	DevOps	35
5.1	DVCS workflow	35
5.2	Release automation	35
5.2.1	Conventional commits	35
5.2.2	Semantic release	35
5.3	Quality assurance	35
5.3.1	Husky and Commitlint	35
5.3.2	Linting and formatting	36
5.4	GitHub Actions workflows: CI and CD	36
5.4.1	Continuous Integration (CI) pipeline	36
5.4.2	Continuous delivery and versioning (<code>release.yml</code>)	36
5.5	Dependency management	36
5.6	Local development environment	37
6	License	37
7	Deployment	37
7.1	Container architecture	37
7.2	Multi-stage builds	38
7.3	Setup and execution	38

1 Analysis

1.1 Problem analysis

1.1.1 Context

Social deduction games have seen a massive surge in popularity, providing engaging multi-player experiences built around communication, deception, and deduction. In parallel, the demand for easily accessible, cross-platform web games that require no client installation has grown significantly.

1.1.2 Problem statement

Developing a real-time multiplayer game in a browser environment introduces severe architectural challenges. Traditional monolithic architectures struggle to manage the dual nature of such applications: on one hand, handling persistent state and RESTful authentication (typically IO-bound); on the other, processing low-latency, real-time game loops and WebSocket connections (often CPU-bound or requiring high concurrency). Coupling these concerns leads to systems that are hard to scale, test, and maintain.

1.1.3 Proposed solution

We propose *The Impostor*, a real-time web-based social deduction game. To address the architectural challenges, the system will be built using a microservices architecture tailored to specific workloads: a robust JVM-based service for stateful user management (lobby-service), a high-concurrency Go service for the core game loop (game-engine), and a Node.js Gateway for real-time bidirectional communication (comm-service).

1.1.4 Roles

The system identifies the following interacting roles:

- **Guest User:** he can join or create rooms.
- **Authenticated User:** he can log in and save his scores onto the DB.
- **Host:** a user who creates a room, holding the privilege to change game settings and start the match.
- **Player:** an active participant in a running match.

1.2 Requirements analysis

1.2.1 System requirements

- **Client environment:** a modern web browser with WebSocket, WebRTC and ES6 JavaScript support (Chrome, Firefox, Safari).
- **Network environment:** a stable broadband internet connection with low latency for real-time event synchronization.
- **Server environment:** a host machine with Docker and Docker Compose installed, necessary for the containerized deployment of the three microservices (lobby-service, game-engine, comm-service).

1.2.2 Functional requirements

The player can:

- Participate anonymously with a random generated **username**.
- Register with unique **username**, **email** and **password** to create a persistent profile, or log into an existing account.
- Leave the **Room** before the start.
- Change game settings and start the game (only the host player).

The system shall:

- Allow an host player to create a private **Room**, generating a unique **Room Code**. Both modes (guest and authenticated) allow full participation in the game.
- Allow users to join a **Room** using its specific **Room Code**.
- Display the list of players in the room in real-time.
- Start a game with a minimum of 4 players and a maximum of 8 players.
- Automatically assign roles (*Crewmate*, *Impostor*).
- Give to all player the **Secret Word** or the **Hint** upon match start.
- Enforce a turn-based phase where players submit their **Clues**.
- Display a **Timer** (each player has a maximum amount of time to submit the **Clue**, once the turn timer expires, the next player automatically moves on).
- Provide a voting mechanism to deduce and eliminate a player at the end of each turn. The player with the most votes is eliminated (in case of draw nobody is eliminated).
- Provide a chat in which players can exchange text messages in real-time (during the entire game) and communicate by voice via WebRTC (only during the discussion phase). Chat is limited to the room you belong to.
- Give the *Impostor* a last chance to guess the **Secret Word**, if he is eliminated.
- Determine the winner:
 - *Impostor* wins if:
 - * He's caught but he finally guesses the **Secret Word**.
 - * He manages to perfectly deceive the other players, making them all eliminate and remaining 1vs1 with a *crewmate*.
 - *Crewmates* win otherwise.

1.2.3 Non-functional requirements

- The architecture must allow independent scaling of the communication layer (Node.js) and the game logic (Go).
- In-game actions (turn changes, voting) must be propagated to clients in real-time via WebSockets.
- The build and deployment process must be fully automated and containerized (Docker, Gradle).
- The architecture must be micro services with three distinct runtimes (JVM, Go, V8).
- The communication between services occurs exclusively via REST/JSON.
- Docker files must be multi-staged and optimized.

1.3 Domain model

1.3.1 Ubiquitous language

To ensure clear communication across the team and consistency in the codebase, the following terms are strictly defined:

Lobby Context

- **Authenticated Player:** a registered user in the system .
- **Anonymous Player:** a guest user in the system.
- **Room:** the waiting area where **Players** gather before a game begins.
- **RoomCode:** the unique alphanumeric code required to join a **Room**.
- **GameHistory:** represents the complete history of a game session in a room.

Engine Context

- **Game:** the aggregate root representing the active game session. It manages the internal state machine, the player list, the secret word, and the flow of domain events.
- **Player:** the logical representation of a participant within the Game, characterized by a status (Alive/Dead) and a Role.
- **Role:** the hidden identity assigned to a Player (**Crewmate** or **Impostor**), which determines their specific win condition.
- **Secret Word:** the specific word, randomly selected by the Engine's internal dictionary, that Crewmates must implicitly describe and the Impostor must deduce.
- **Hint:** a broad category or clue related to the Secret Word, automatically assigned by the Engine to provide context to the players.

- **Turn / Phase:** the current state of the game loop, characterized by a timer and a specific Phase (`DISCUSSION`, `VOTING`, or `GUESSING_WORD`). It dictates which actions are currently permitted.
- **Vote:** the action performed during the Voting phase where an alive Player points to another Player as the suspected Impostor, or opts to abstain (`SKIP`).
- **Guessing Attempt:** the action performed by an eliminated Impostor during the `GUESSING_WORD` phase, representing their final chance to guess the Secret Word and steal the victory.

Comm Context

- **Client:** the application connected to the system.
- **Connection:** the bidirectional WebSocket tunnel.
- **Session:** the logical link associating a **Connection** to a User ID.
- **Message:** the aggregate information for trigger an event.
- **Event:** a validated message representing an action or state update.

1.3.2 Bounded context

The architectural system is divided into three main Bounded Contexts, mapping directly to our microservices:

- **Lobby Context (Java/Spring):** it manages the pre-game aspects. It has no knowledge of the game rules.
- **Engine Context (Go):** the *Core Domain*. It contains the business logic. It is agnostic regarding real user identities, managing only players.
- **Comm Context (Node.js):** it acts as an infrastructural *Gateway* (BFF), routing WebSocket events to the downstream backend services.

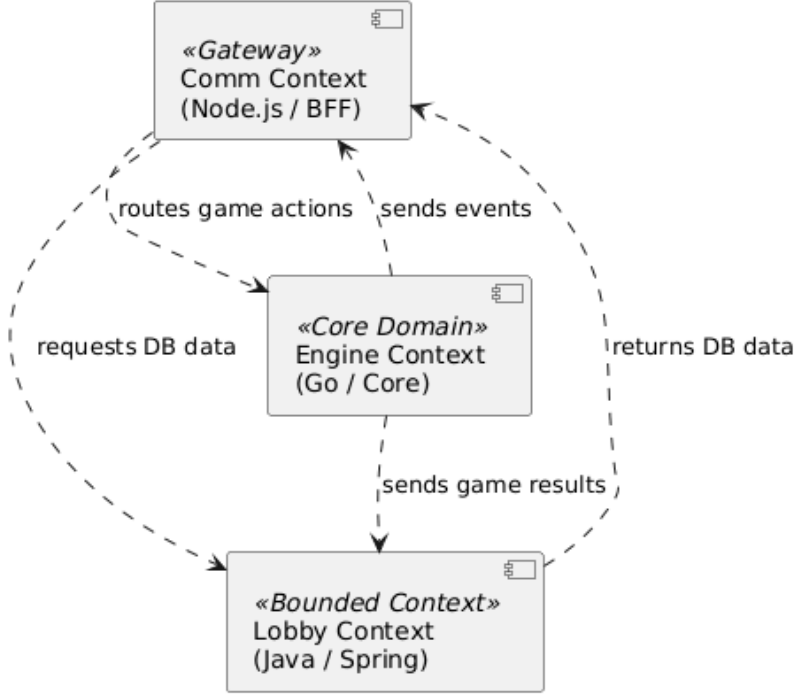


Figure 1: Bounded Context

2 Design

In this section, we define the software design of The Impostor, focusing on how the system’s microservices interact to fulfill the identified functional and non-functional requirements. Adopting a Domain-Driven Design (DDD) approach, the system’s complexity is broken down into three autonomous Bounded Contexts: **Lobby**, **Engine** (*Core Domain*), and **Comm** (*Gateway*). This decomposition allows for a strict separation of concerns, enabling a polyglot architecture (Java, Go, Node.js) where each context manages its own specific domain logic, state, and data integrity. The following sections explore the tactical design of these contexts detailing their aggregates, entities, and value objects and illustrate the communication patterns that tie the distributed architecture together.

2.1 Engine Context

This context is the heart of the system, serving as the definitive source of truth for the game rules, state progression, and win conditions. Its domain model is centered around the **Game Aggregate**, which ensures that all business invariants are strictly enforced: for instance the votes are solely accepted during the designated phase, and the minimum player count is respected. It:

- Encapsulates the core state.
- Manages **Player** entities and assigns value objects such as **Role** (*Crewmate/Impostor*), **Secret Word**, **Hint** and **Turn**.

To instantiate this model, the context utilizes a **GameFactory** that processes initialization data provided by the upstream **Lobby Context**. This acts as a protective boundary, ensuring that the Engine remains completely agnostic to external user identities, database

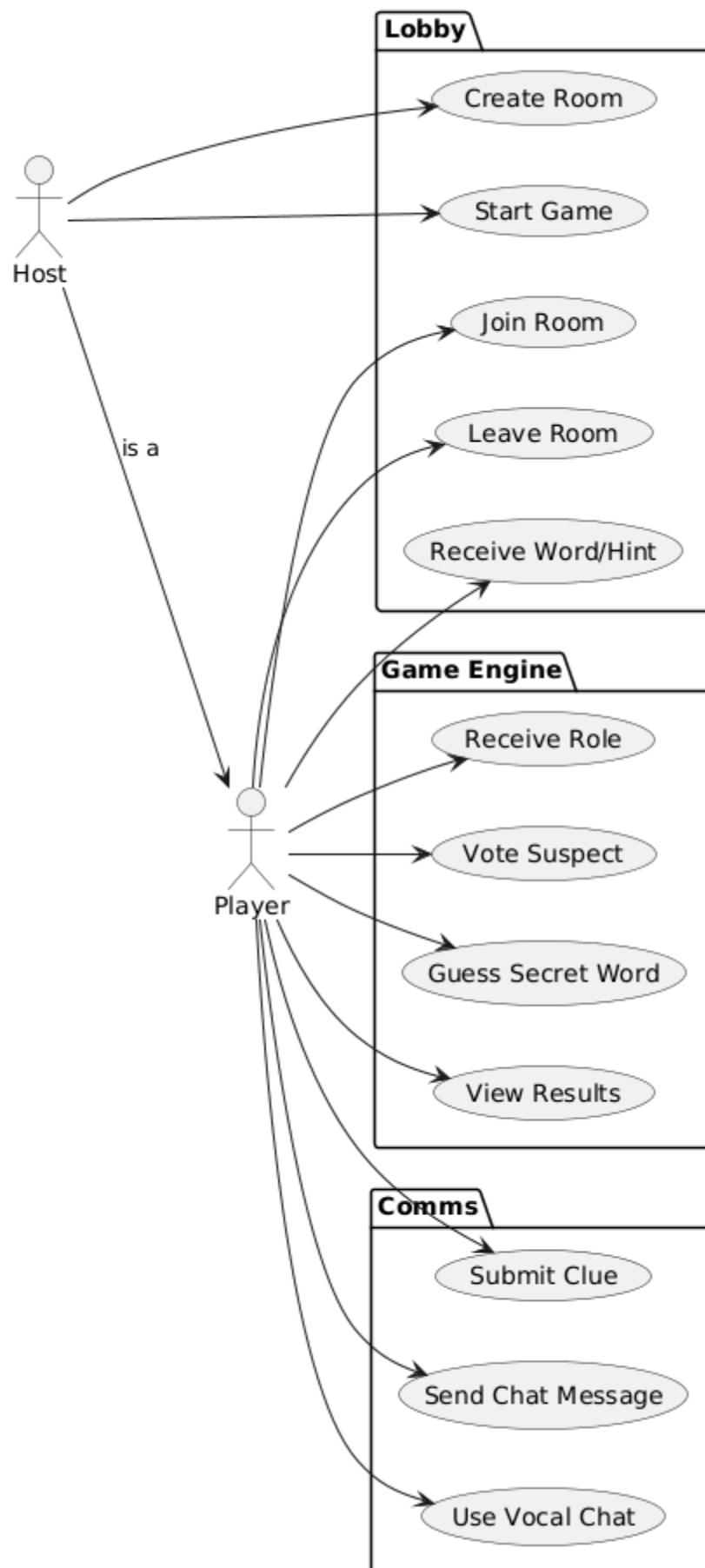


Figure 2: Use cases schema

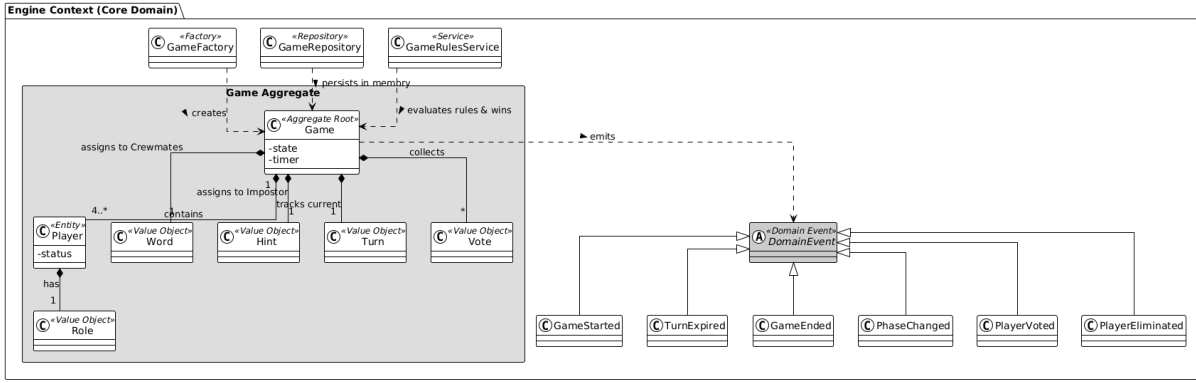


Figure 3: Engine Context

specifics, or network protocols, focusing solely on the logical representation of the match.

The context relies on critical domain services, most notably the `GameRulesService`. This stateless component orchestrates the complex logic of the game loop, such as calculating vote outcomes (gracefully handling draws), evaluating the final guess of an eliminated impostor, and determining if the 1 vs 1 victory condition has been met.

Finally, whenever the state of the game changes such as a player casts a vote, the timer is about to expire or the match ends, the context publishes domain events like `TurnExpired` or `GameEnded`. This event-driven approach enables the **Comm Context** (*Gateway*) to react asynchronously, broadcasting real-time WebSocket updates to the clients without tightly coupling the core game logic to the network infrastructure.

2.2 Lobby Context

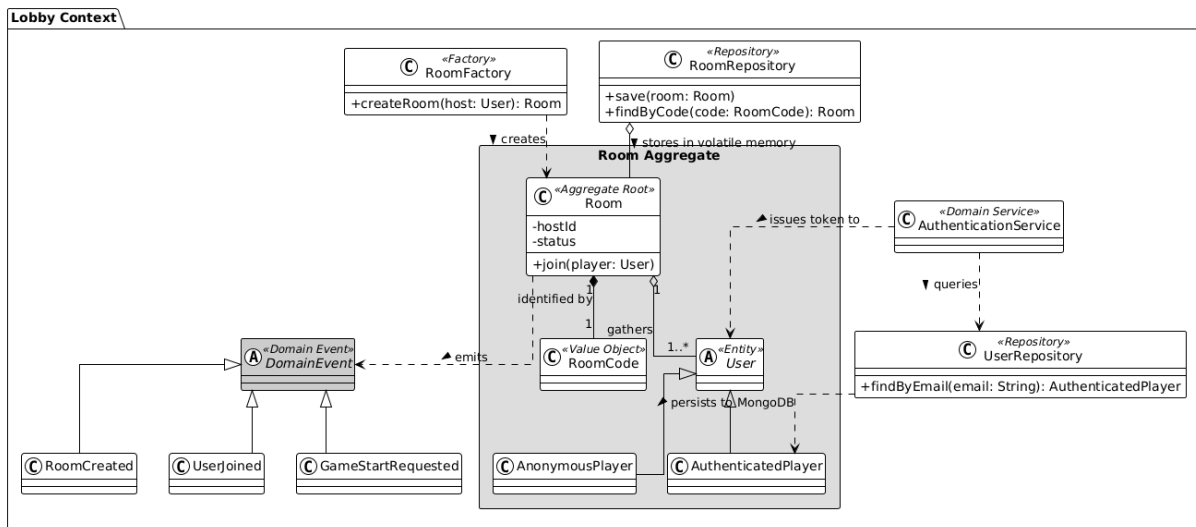


Figure 4: Lobby Context

This context represents the entry point of the system, responsible for identity management, security, and the orchestration of pre-game waiting rooms. The tactical design

is centered around the **Room Aggregate**, which maintains the integrity of the match-making phase, ensuring that business invariants such as the unique nature of usernames within a session and the minimum player count required to start are strictly enforced.

The persistence strategy is bifurcated to optimize for both durability and performance. A **UserRepository** is employed to interface with a MongoDB instance, handling the persistent lifecycle of **AuthenticatedPlayer** profiles and their credentials.

Conversely, the **RoomRepository** operates exclusively in-memory, acting as a high speed collection for active **Room Aggregate(s)**. This architectural choice reflects the ephemeral nature of waiting rooms: they are volatile entities that do not require long-term storage, thus reducing database overhead and latency.

To handle the diversity of participants, the model adopts a polymorphic approach through the **User** entity, which is specialized into **AuthenticatedPlayer** (for registered users) and **AnonymousPlayer**.

The **AuthenticationService** acts as a Domain Service that orchestrates the verification of credentials and the issuance of security tokens, providing a unified identity interface for the downstream services.

The lifecycle of a room is managed by the **RoomFactory**, which encapsulates the complexity of generating unique **RoomCode** Value Objects and initializing the aggregate state. Once the host triggers the transition to the playing phase, the aggregate publishes a **GameStartRequested** Domain Event. This event serves as the decoupled synchronization mechanism that carries the necessary payload to the **Engine Context**, effectively handing over the control flow while maintaining strict boundary isolation between the social and the logic driven domains of the application.

2.3 Comm Context

The **Comm Context** functions as the system's infrastructural *Gateway and Backend For Frontend (BFF)*, serving as the single point of entry for all client-side real-time interactions. Developed in Node.js to leverage its non-blocking I/O model, this context is specifically designed to handle the high concurrency required by persistent WebSocket connections and the low-latency demands of WebRTC signaling.

Architecturally, the context is centered around the **Session Aggregate**, which manages the mapping between a logical **UserID** and a physical **SocketID**. Given the transient nature of network sessions, the **SessionRepository** implements an in-memory storage strategy, allowing for sub-millisecond lookups and ensuring that resources are immediately reclaimed upon client disconnection.

The context provides two primary specialized services:

- **RoutingService**: acts as an intelligent proxy that decouples the frontend from the internal microservices. it translates incoming WebSocket events into upstream

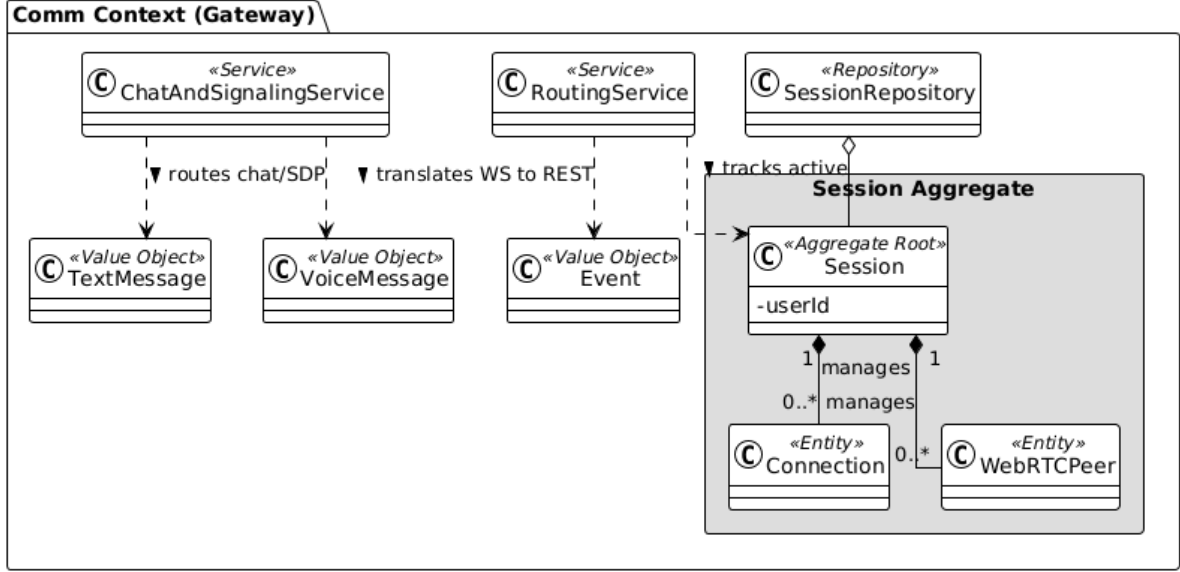


Figure 5: Comm Context

REST/JSON requests towards the Lobby or Engine contexts, shielding the core logic from the complexities of the transport layer.

- **ChatAndSignalingService:** manages the cross-cutting concerns of real-time communication. It facilitates the exchange of *Session Description Protocol (SDP)* and *Interactive Connectivity Establishment (ICE)* candidates for WebRTC peer-to-peer audio, while also orchestrating the broadcasting of text-based messages within specific room channels.

By isolating communication concerns within this dedicated context, the system achieves a superior degree of scalability. The I/O-heavy gateway can be scaled independently from the CPU-heavy Engine, ensuring that the voice chat and real-time responsiveness remain stable even as game logic complexity increases.

2.4 Mockups

We have created a series of mockups for *The Impostor* user interface design that illustrate the key screens and user interactions. These mockups aim to provide players with a clear and intuitive experience, ensuring that the transition between game phases is fluid and engaging. Our design follows a cross-platform approach, providing a consistent "flat" aesthetic optimized for both desktop and mobile devices.

The mockups include the following screens, representing the logical progression from user onboarding to active gameplay:

- **Login and Registration:** an entry point where users can create a new account, log into an existing one or enter as guest.

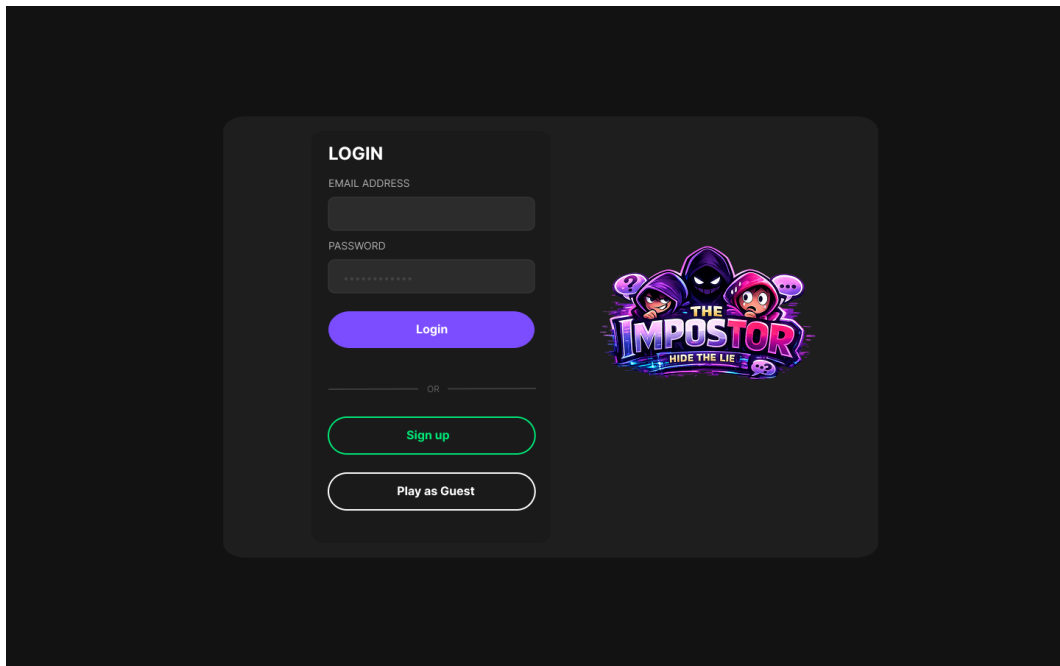


Figure 6: Login page - Desktop view

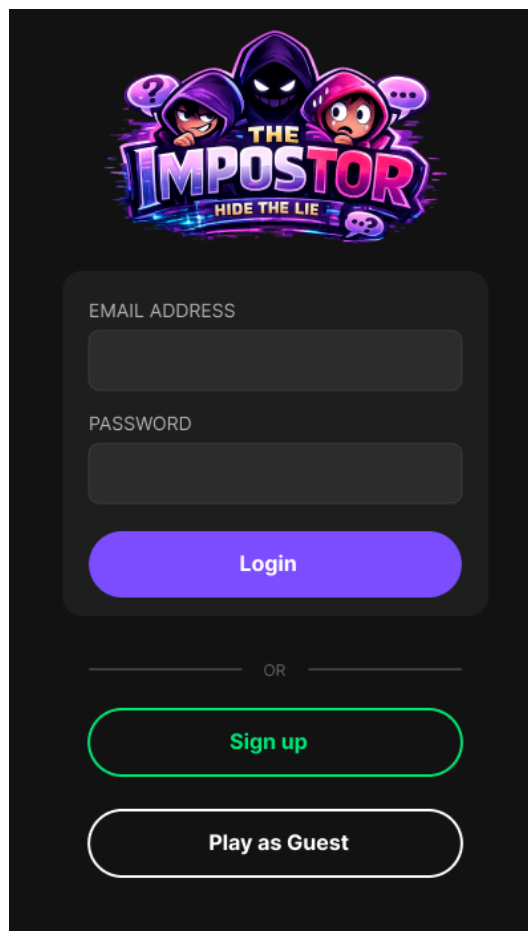


Figure 7: Registration page - Mobile view

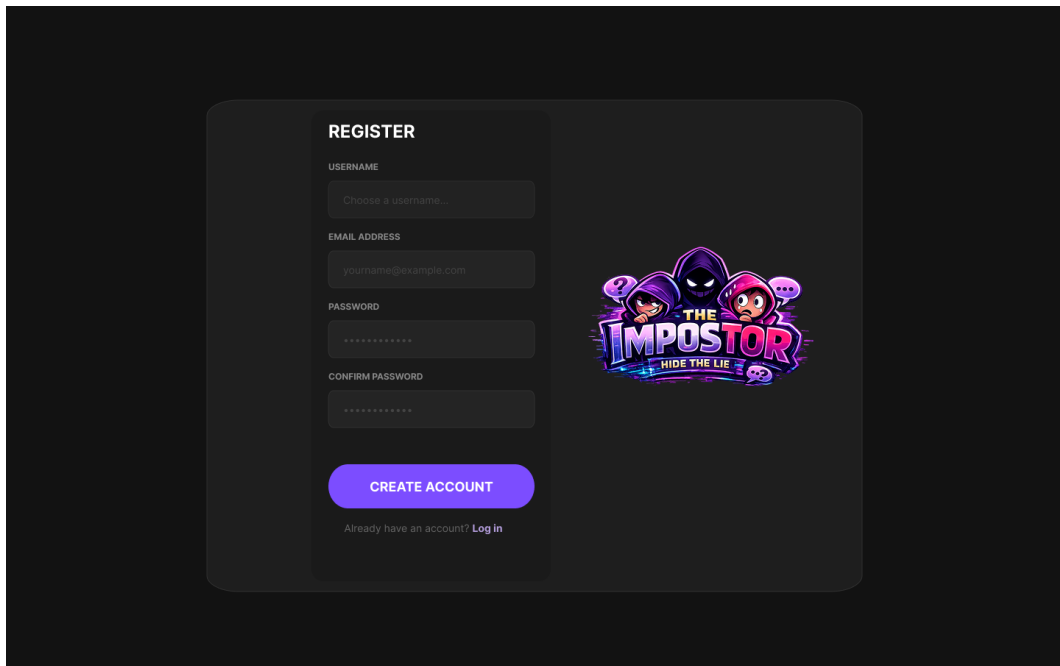


Figure 8: Registration page - Desktop view



USERNAME

EMAIL ADDRESS

PASSWORD

CONFIRM PASSWORD

CREATE ACCOUNT

[Already have an account? Log In](#)

Figure 9: Registration page - Mobile view

- **Home Page:** after authentication, this hub allows players to create a new game lobby as a host or join an existing one by entering a unique room code.

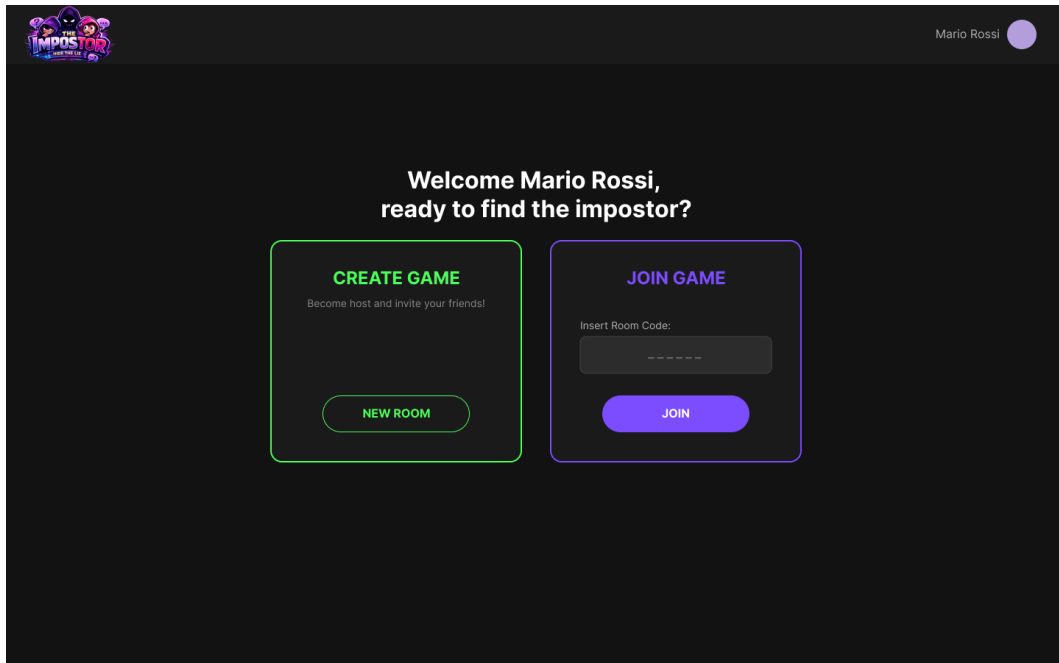


Figure 10: Home page - Desktop view

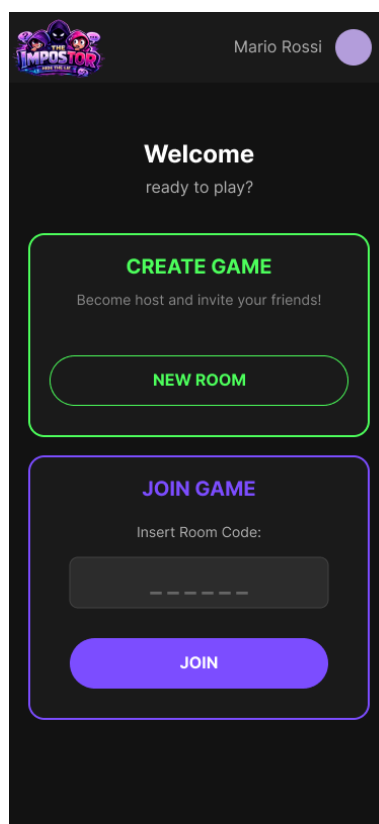


Figure 11: Home page - Mobile view

- **Lobby Page:** an interactive waiting area where players gather before the match begins. It displays the list of connected participants and their "Ready" status. For the host, this page includes the controls to adjust game settings and the "Start Game" trigger to initiate the match.

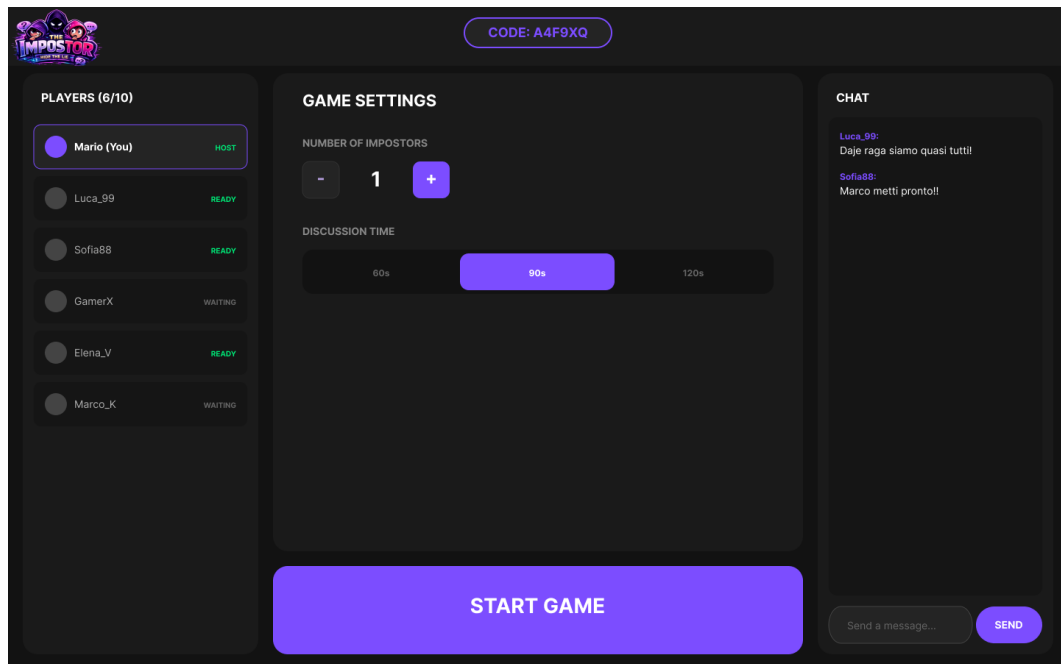


Figure 12: Lobby page - Desktop view

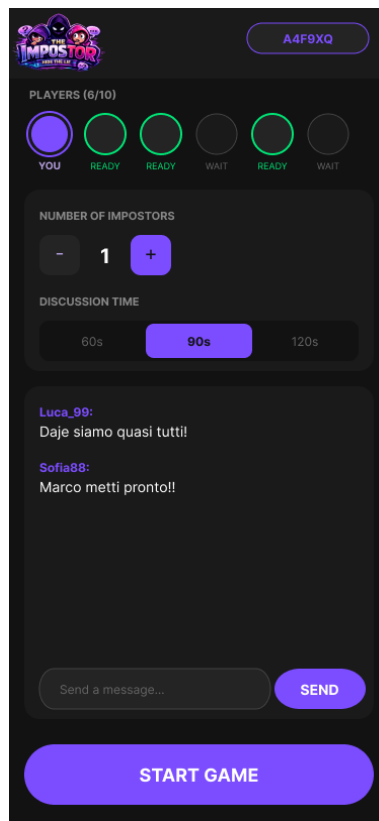


Figure 13: Lobby page - Mobile view

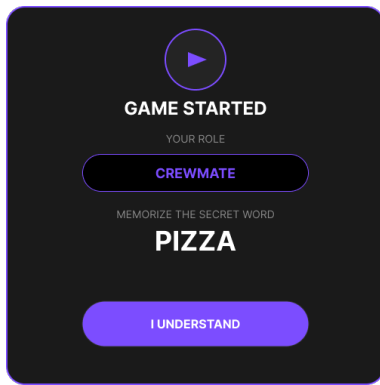


Figure 14: Secret word - Crewmate view

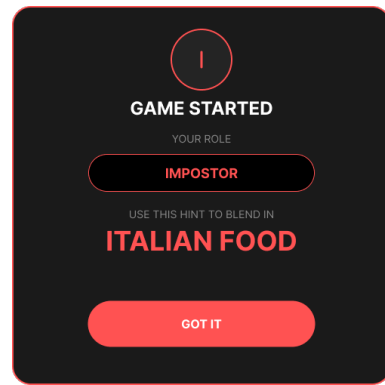


Figure 15: Hint panel - Impostor view

- **Gameplay Page:** the core dashboard of the game session. This dynamic page integrates all the essential elements of the match: the display of the secret word or hint, the history of terms submitted by players, a live chat for discussion, and the interactive voting system to identify the impostor.

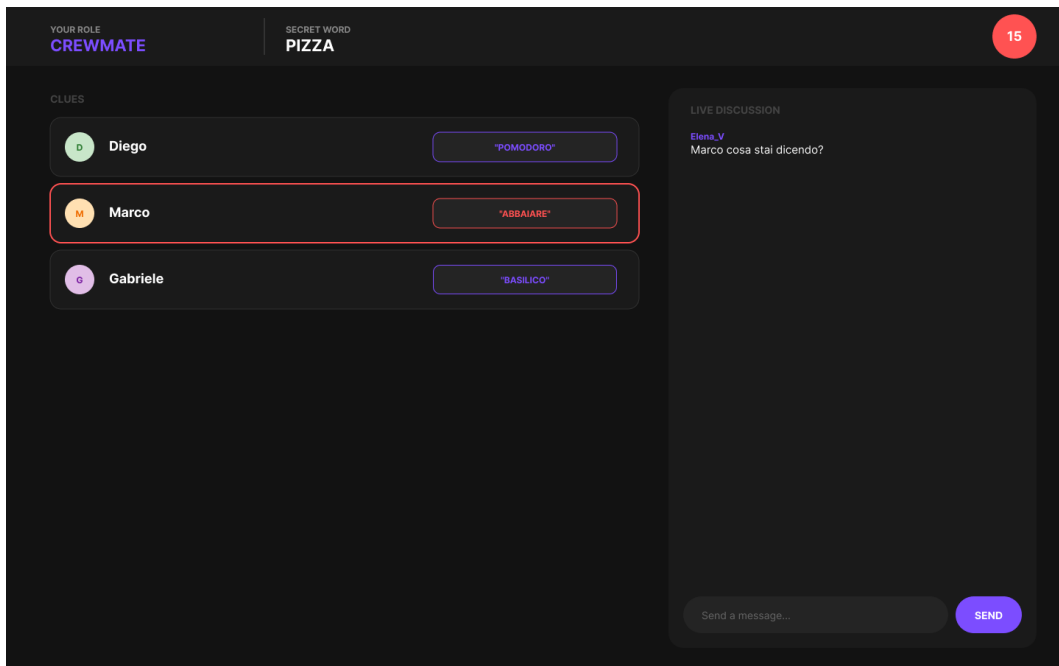


Figure 16: Gameplay page - Desktop view

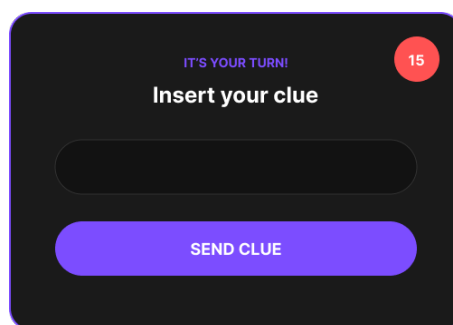


Figure 17: Insert clue panel

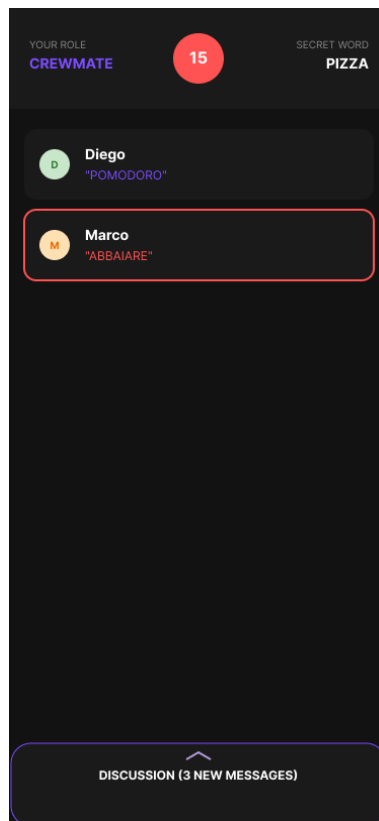


Figure 18: Gameplay page - Mobile view

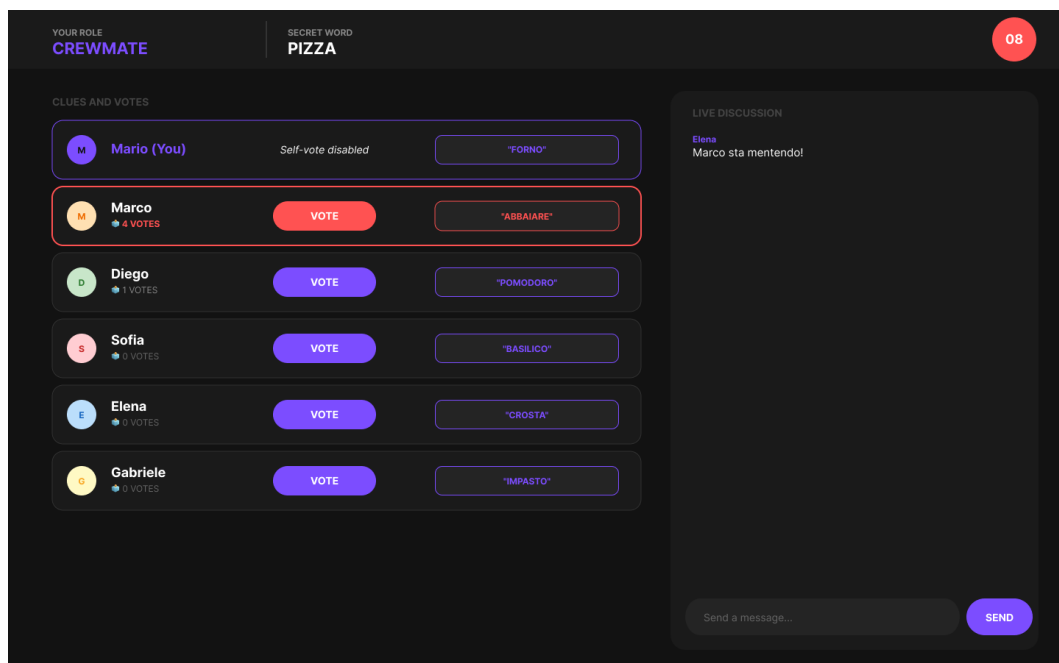


Figure 19: Vote page - Desktop view

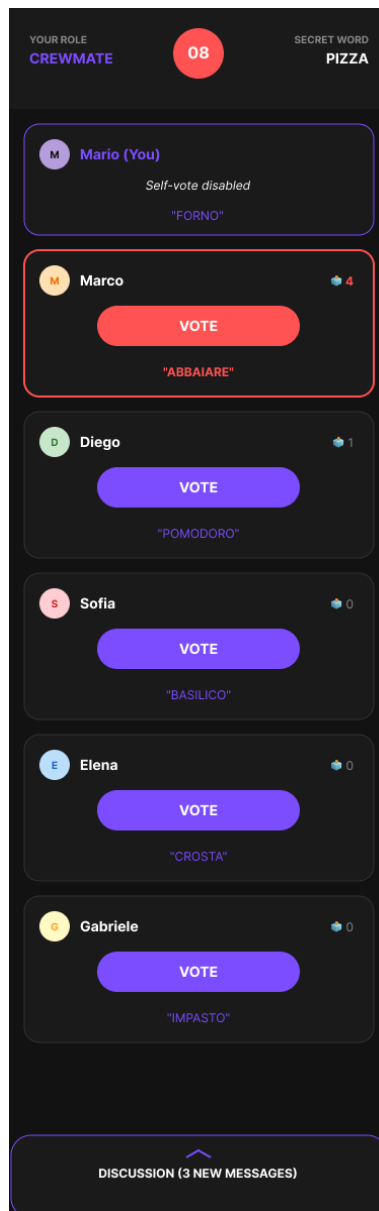


Figure 20: Vote page - Mobile view

3 Architecture

3.1 Physical architecture

The system is a distributed Client-Server application based on a microservices architecture, designed to run in a containerized environment (e.g., Docker). The entire backend ecosystem is structured as a monorepo orchestrated by **Gradle**, which handles the build automation, task execution, and dependency management across the different technology stacks. The system is composed of the following main components:

- **Client (front-end):** a Single Page Application (SPA) running in the user's browser or mobile device. It communicates with the backend ecosystem using standard HTTP REST APIs for onboarding operations, and WebSockets/WebRTC for real-time synchronization and voice interactions.
- **Lobby & Account Service (back-end):** built with **Java Spring Boot**. It handles user authentication, session management, room creation, and game configurations. It represents the primary business entry point for players before a match begins, leveraging the Spring ecosystem for robust REST API design and security.
- **Game Engine (back-end):** a high-performance state manager built with **Go**. Its sole responsibility is executing the core game logic, handling round timers, and validating player turns.
- **Communications Gateway (back-end):** built with **Node.js**. It acts as a unified entry point and real-time hub, routing client requests to backend services and managing persistent connections. It manages persistent WebSocket connections to broadcast chat messages and game state updates, and handles WebRTC signaling for the live discussion phase.
- **Database:** a noSQL document database (**MongoDB**) used primarily by the Java Spring service. It securely writes and reads user credentials, player statistics, and persistent lobby configurations.

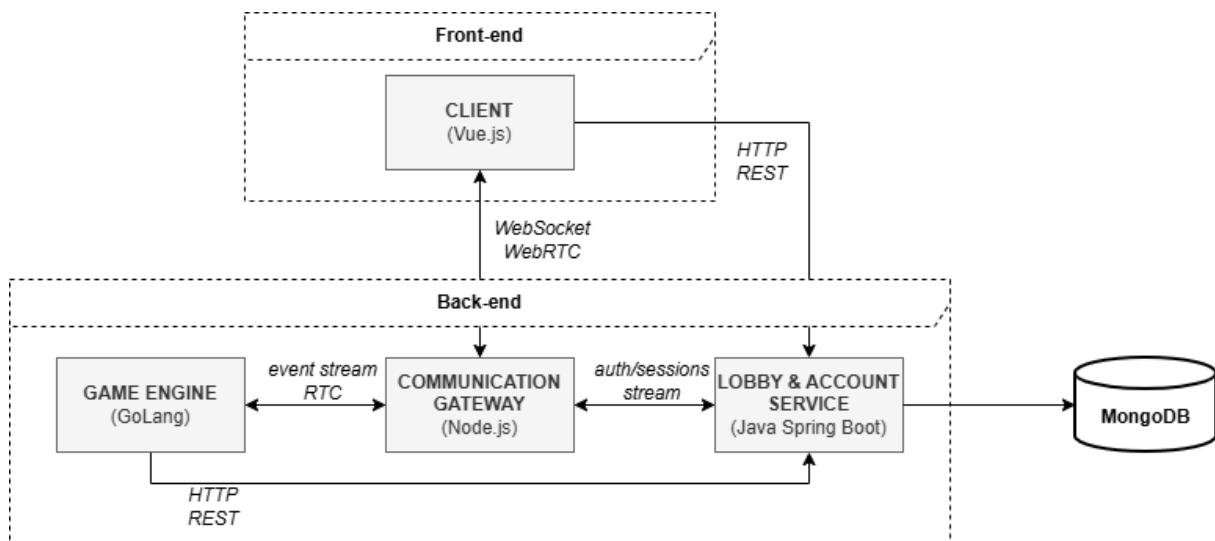


Figure 21: Physical architecture

3.2 Logical architecture

Within each specific Bounded Context (such as the Lobby Service and the Game Engine), we applied **Hexagonal Architecture** (Ports and Adapters) principles to protect the core Domain Model from technological and framework coupling. This architectural style is particularly crucial for isolating the internal logic (game rules) from the delivery mechanisms (HTTP, WebSockets).

Each microservice is conceptually divided into three concentric layers:

- **Domain Layer (Inner Layer):** contains the core Entities, Value Objects, and business invariants (**Player**, **Room**, **GameState**, **Vote**). It is entirely free of external dependencies, database frameworks, or network libraries. This is where the interfaces (Ports) for external services and persistence are strictly defined.
- **Application Layer (Middle Layer):** contains the Domain Services and Command Handlers that implement the specific use cases of the game (**CreateLobbyUseCase**, **SubmitClueUseCase**, **CastVoteUseCase**). It coordinates the data flow to and from the domain entities without knowing the specific details of the technical implementations.
- **Infrastructure Layer (Outer Layer):** contains the concrete implementations (Adapters) of the interfaces defined in the Domain. This includes the Java Spring Boot REST controllers, the Node.js WebSocket event handlers, the database Repositories for **MongoDB**, and the internal HTTP clients used for inter-service communication between Java, Go, and Node.js.

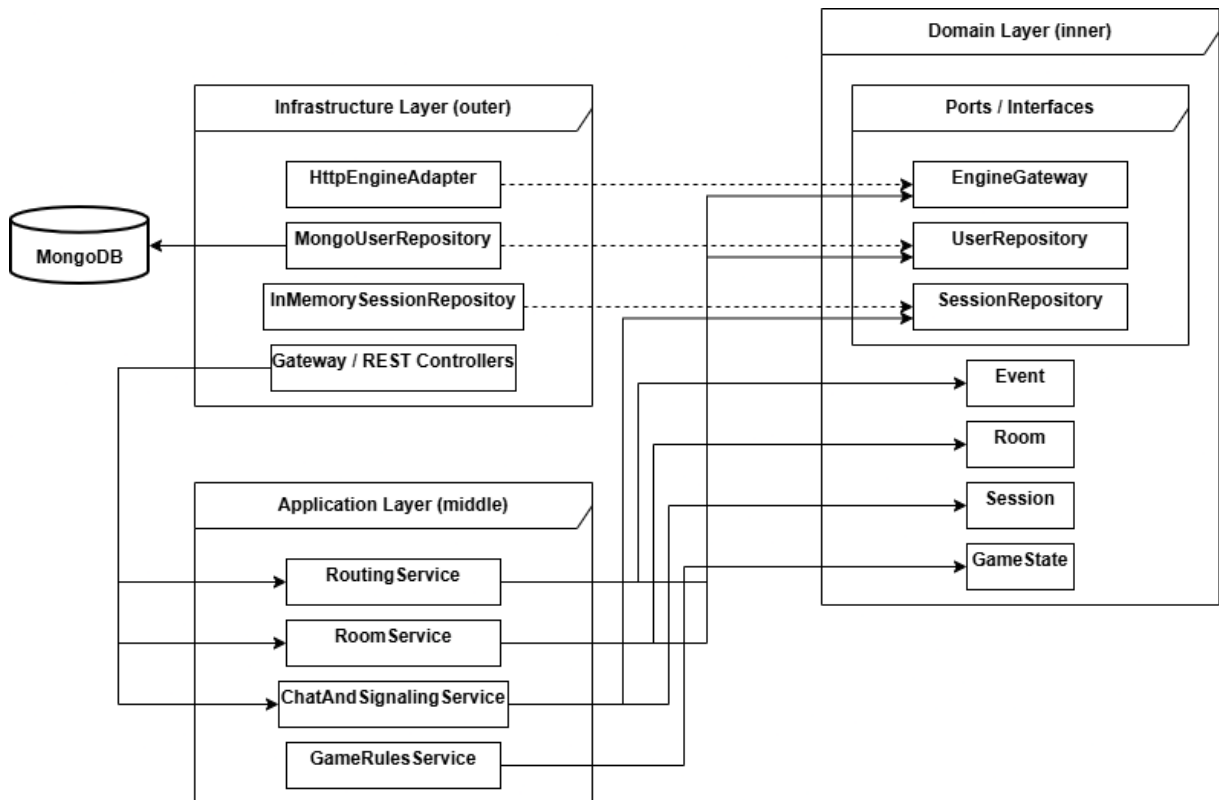


Figure 22: Logical architecture

4 Implementation

4.1 Technologies used

To fulfill the non-functional requirements of scalability, isolation, and maintainability, *The Impostor* project adopts a polyglot microservices architecture[cite: 1]. The choice of the technology stack was not arbitrary; rather, it was driven by the specific workload requirements of each *Bounded Context*:

- **Lobby & Account Service (Java 21 and Spring Boot)**: the Spring Boot framework was chosen to implement the user and pre-game room management service.
- **Database (MongoDB)**: a document-oriented NoSQL database, utilized exclusively by the Lobby Service for the long-term persistence of user profiles, credentials, and aggregated post-match statistics.
- **Game Engine (Go)**: the core domain engine was developed in Go. It lent itself very well to the development of the engine.
- **Communications Gateway (Node.js)**: this service acts as the *Backend For Frontend* (BFF). It leverages the event-driven architecture and the non-blocking I/O model of the Node.js V8 engine.
- **Client (Vue.js)**: the user interface is a *Single Page Application* (SPA) developed with Vue.js. It communicates with the backend ecosystem using standard HTTP REST APIs for onboarding operations and WebSocket/WebRTC tunnels for real-time gameplay synchronization.
- **Infrastructure and Automation**: the entire application ecosystem is containerized using **Docker** and orchestrated locally via **Docker Compose**. Furthermore, the project is structured as a *monorepo* managed by **Gradle**, which automates the build processes across the different technology stacks.

4.2 Game Engine implementation (Go)

The *Game Engine* microservice, developed in Go 1.24, represents the *Core Domain* of *The Impostor* ecosystem. Its exclusive task is to orchestrate the business logic, advance the game's state machine, and evaluate win conditions, maintaining strict isolation from external infrastructural concerns. To achieve this goal, the service was designed following the patterns of *Hexagonal Architecture* and *Domain-Driven Design* (DDD).

4.2.1 The Aggregate Root and the Event-Driven approach

The core of the domain is represented by the **Game** entity, which acts as the *Aggregate Root*. This structure holds the volatile state of the match, the player list, and the current turn phase in memory. Instead of directly invoking network calls to update clients, the domain adopts an *Event-Driven* pattern. Every valid state mutation within the aggregate generates a *Domain Event* (e.g., *PhaseChanged* or *PlayerVoted*), which is temporarily queued within the aggregate itself.

```

1 // internal/domain/aggregate/game.go
2 type Game struct {
3     ID            string
4     State         string
5     Players       []entity.Player
6     CurrentTurn   valueobject.Turn
7     SecretWord    valueobject.SecretWord
8     Votes         []valueobject.Vote
9     domainEvents  []event.DomainEvent
10    ...
11 }
12
13 func (g *Game) AdvanceToVoting() error {
14     // 1. Business invariants validation
15     if g.State != StatePlaying {
16         return errors.New("...")
17     }
18
19     // 2. Internal state mutation
20     g.CurrentTurn.Phase = valueobject.PhaseVoting
21     g.CurrentTurn.Timer = 60
22     g.Votes = make([]valueobject.Vote, 0)
23
24     // 3. Domain event recording
25     g.RecordEvent(event.PhaseChanged{
26         BaseEvent: event.BaseEvent{OccurredAt: time.Now(),
27             },
28         GameID:      g.ID,
29         NewPhase:      string(valueobject.PhaseVoting),
30         Timer:         60,
31     })
32
33     return nil
34 }

```

Listing 1: State and event management in the Game Aggregate

4.2.2 Thread-safety in volatile memory

Since the game engine processes a high volume of concurrent requests (e.g., simultaneous votes cast by players), in-memory persistence (RAM) requires careful management to prevent *race conditions*. The implementation of the `InMemoryGameRepository` leverages Go's native synchronization primitives, specifically `sync.RWMutex`. This approach ensures *thread-safety*, allowing concurrent reads while exclusively isolating writes.

```

1 // internal/infrastructure/memory/gameRepository.go
2 type InMemoryGameRepository struct {
3     mu    sync.RWMutex
4     games map[string]*aggregate.Game
5 }
6

```



```

7 func (r *InMemoryGameRepository) Save(game *aggregate.Game) error
8 {
9     r.mu.Lock()           // Acquire exclusive write lock
10    defer r.mu.Unlock()    // Guaranteed release at the end of
11                           the function
12    r.games[game.ID] = game
13    return nil
14 }

```

Listing 2: Concurrent access synchronization in the in-memory Repository

4.2.3 Asynchronous execution and Goroutines

To meet the low-latency requirements of the *game loop*, the application layer (*GameAppService*) delegates blocking communications towards other microservices to lightweight, concurrent threads called *goroutines*.

At the end of a match, for instance, statistics must be transmitted to the *Lobby Service* to be persisted on MongoDB. By executing the HTTP client within a *goroutine* using a *fire-and-forget* approach, the game engine instantly notifies players of the game's conclusion, without suffering delays caused by network I/O.

```

1 // internal/application/gameAppService.go
2 // ... (winner calculation) ...
3
4 // Composing the DTO with the statistics
5 roundResult := LobbyRoundResult{
6     RoomCode:      gameID,
7     ImpostorID:     impostorID,
8     WinnerID:       winnerID,
9     ImpostorWon:    winTeam == service.WinImpostors,
10    SecretWord:     string(game.SecretWord),
11    // ...
12 }
13
14 // Asynchronous execution of the network call to the Lobby
15 // Service
16 go func() {
17     app.lobbyGateway.SaveRoundResult(roundResult)
18 }()
19
20 // Instant WebSocket notification via Webhook
21 app.notifier.NotifyEvent("GameEnded", map[string]interface{}{
22     "gameId": gameID,
23     "winner": string(winTeam),
24 })

```

Listing 3: Using goroutines for non-blocking network communication

4.3 Client implementation (Vue.js)

It's a *Single-page application* built with Vue 3 + Vite, responsible for the user interface and player experience (rendering, input, animations, voice chat).

Presentation logic is kept separate from domain decisions, which are handled by the *Game Engine* and *Lobby Service*. It communicates with the *Comm Service* using event envelopes (type + payload) and it uses a REST client for *Lobby Service* calls (auth, create/join/leave room).

4.3.1 State management (Pinia)

Pinia is used as the single source of truth for game state: the room store centralizes players, gamePhase, votes, messages and UI flags. Updates originate from local actions (submit clue, vote) and from realtime events. Actions apply idempotent state changes, trigger side effects (API calls, RTC) and expose derived getters for the UI.

```
// src/stores/room.js
export const useRoomStore = defineStore('room', () => {
  ...
  // handle incoming WebSocket messages
  function handleIncoming(msg) {
    const type = msg.type || (msg.payload && msg.payload.type)

    if (type === 'room_update') players.value = msg.payload?.players || []
    ...
    // add a system message when a player votes
    if (type === 'PlayerVoted') messages.value.push({ system: true, text:
      'Vote: ${msg.payload?.voterId} ${msg.payload?.targetId}' })
    ...
  }

  function initWS(code) {
    roomCode.value = code
    wsService.connect()
    // identify current user if available
    if (authStore.user?.id) {
      wsService.send({ type: 'IDENTIFY', payload: { userId: authStore.user.id
        } })
    }
    if (unsubscribe) unsubscribe()
    // subscribe to incoming messages
    unsubscribe = wsService.onMessage(handleIncoming)
  }

  // join a room
  async function join(code, displayName) {
    loading.value = true
    try { // call API
      await roomService.joinRoom(code, displayName)
    } catch { ... }
    initWS(code)
  }
}
```

```

    // notify server about joining
    wsService.sendEvent('join_room', { roomCode: code, username: displayName })
    loading.value = false
  }
  ...
  // send a chat message
  function sendChat(text) {
    ...
    wsService.send({ type: 'CHAT', roomId: roomCode.value, payload: { content:
      text } })
    // add message locally
    messages.value.push({ sender: 'me', text })
  }
  ...

```

Listing 4: Room state management and WebSocket integration using Pinia

4.3.2 Real-time communication (WebSocket)

Real-time is handled by a dedicated WebSocket service that exchanges event envelopes of the shape { type, payload }. The service exposes connect, sendEvent and a listener registration API. Incoming events are mapped to store actions and UI updates, while outgoing user actions are serialized and sent to the *Comm Service*.

```

// src/services/wsService.js

// create and setup WebSocket connection
function connect() {
  ...
  socket = new WebSocket(WS_BASE)
  ...
  // handle incoming messages
  socket.onmessage = (ev) => {
    try {
      const parsed = JSON.parse(ev.data)
      // notify all listeners
      listeners.forEach((cb) => cb(parsed))
    } catch (e) { ... }
  }
  ...
}

...
// send a raw message object via WebSocket
function send(obj) {
  ...
  const payload = JSON.stringify(obj)
  if (socket.readyState === WebSocket.OPEN) {
    socket.send(payload)
  } else {
    // wait for connection to open, then send once
    socket.addEventListener('open', () => socket.send(payload), { once: true })
  }
}

```

```

    }
  }

  // send a standardized event message
  function sendEvent(action, payload = {}) {
    send({ type: 'EVENT', payload: { action, ...payload } })
  }

  // subscribe to incoming messages
  function onMessage(cb) {
    listeners.push(cb)
    // return unsubscribe function
    return () => {
      listeners = listeners.filter((c) => c !== cb)
    }
  }
}

```

Listing 5: WebSocket service implementation and real-time event broadcasting

4.3.3 Voice chat (WebRTC)

Audio is implemented as a P2P mesh via an encapsulated service: it manages RTCPeerConnection instances, localStream, STUN/ICE configuration and uses the WS channel for signalling (offer/answer/ICE). The service handles peer lifecycle, mute/unmute and cleanup to avoid resource leaks.

```

// src/services/voiceChatService.js

// initialize WebRTC for the room
async function init(room, userId, peerIds) {
  try {
    // get local audio stream (microphone only)
    localStream = await navigator.mediaDevices.getUserMedia({ audio: true,
      video: false })
  } catch (e) {...}
  ...
  // listen for signaling messages
  unsubscribe = wsService.onMessage(_handleSignal)
  // create offers only for peers with higher IDs
  for (const peerId of peerIds) {
    if (peerId !== myId && myId < peerId) { await _createOffer(peerId) }
  }
}
...
// handle incoming WebRTC signaling messages
function _handleSignal(msg) {
  ...
  const { senderId, to, signal } = msg.payload || {}
  ...
  // route signal by type
  if (signal.type === 'offer') { _handleOffer(senderId, signal).catch(...) }
}

```

```

    else if (signal.type === 'answer') { _handleAnswer(senderId,
        signal).catch(...)
    }
    else if (signal.type === 'ice-candidate') {
        _handleIceCandidate(senderId, signal.candidate).catch(...)
    }
}
// get or create a peer connection
function _getOrCreatePeerConnection(peerId) {
    ...
    // create new RTCPeerConnection
    const pc = new RTCPeerConnection({ iceServers: ICE_SERVERS })
    // add local tracks to connection
    if (localStream) {
        localStream.getTracks().forEach(track => pc.addTrack(track, localStream))
    }
    // send ICE candidates to peer
    pc.onicecandidate = ({ candidate }) => {
        if (candidate) {
            _sendSignal(peerId, { type: 'ice-candidate', candidate })
        }
    }
    // play incoming audio stream
    pc.ontrack = ({ streams }) => {
        const audio = new Audio()
        audio.srcObject = streams[0]
        audio.autoplay = true
    }
    // remove peer on failure/close
    pc.onconnectionstatechange = () => {...}
    peers.set(peerId, pc)
    return pc
}

// create and send an offer to a peer
async function _createOffer(peerId) {
    const pc = _getOrCreatePeerConnection(peerId)
    const offer = await pc.createOffer()
    await pc.setLocalDescription(offer)
    // send offer SDP
    _sendSignal(peerId, { type: 'offer', sdp: offer.sdp })
}

// send signaling message via WebSocket
function _sendSignal(to, signal) {
    wsService.send({
        type: 'WEBRTC',
        payload: { roomId: roomCode, to, signal }
    })
}

```

Listing 6: WebRTC voice chat initialization and peer-to-peer signaling management

4.3.4 Backend integration (REST API)

Synchronous backend calls (auth, create/join/leave room, fetch data) use `apiClient.js`, which reads the Lobby Service API URL from `.env` and standardizes JSON headers, error handling and credentials. Higher-level wrappers like `roomService.js` and `authService.js` encapsulate endpoints so views and stores only call plain service methods.

```
// src/services/apiClient.js

// return authorization header (bearer token) from localStorage
function authHeader() {
  const token = localStorage.getItem('token');
  return token ? { Authorization: `Bearer ${token}` } : {};
}

// perform HTTP request to the API base
async function request(path, { method = 'GET', body = null } = {}) {
  const url = path.startsWith('/') ? `${API}${path}` : `${API}/${path}`;
  const headers = { 'Content-Type': 'application/json', ...authHeader() };
  const opts = { method, headers };
  if (body !== null && body !== undefined) {
    opts.body = typeof body === 'string' ? body : JSON.stringify(body);
  }

  const res = await fetch(url, opts);
  const ct = res.headers.get('content-type') || '';
  const data = ct.includes('application/json') ? await res.json().catch(() =>
    null) : await res.text().catch(() => null);

  if (!res.ok) {
    ...
    throw err;
  }
  return data;
}

export const apiClient = {
  get: (path) => request(path, { method: 'GET' }),
  post: (path, body) => request(path, { method: 'POST', body }),
  put: (path, body) => request(path, { method: 'PUT', body }),
  delete: (path) => request(path, { method: 'DELETE' })
};

export default apiClient;
```

Listing 7: Centralized API client wrapper with automatic JWT authorization and request parsing

4.4 Gateway implementation

The Comm Context (`comm-service`) has been implemented using Node.js and TypeScript. This technology stack was specifically chosen for its asynchronous, event-driven, and non-blocking I/O model, making it highly suitable for handling a massive number of concurrent WebSocket connections with minimal latency. The service acts as the central nervous system of the architecture, decoupling the client-side application from the internal backend microservices.

The implementation is structured around four distinct core responsibilities:

4.4.1 Real-time lobby synchronization

Through the `LobbyService.ts` module, the gateway acts as a synchronization bridge between the client and the Java-based Lobby Service. Whenever a pre-game action occurs—such as a player joining a room, toggling their "ready" state, or the host altering game settings—the gateway forwards the state mutation via REST to the Java service. Furthermore, this module securely fetches the unique secret word from the Lobby database right before initializing a new match, injecting it into the payload for the engine:

```
1 if (eventType === 'start_game' || eventType === 'START_GAME') {
2   const hostId = payload.hostId || currentUserId;
3   const playerIds = await lobbyService.handleStartGame(roomCode,
4     hostId) || [];
5
6   // Fetch the secret word from the Java Lobby DB
7   const wordEntry = await lobbyService.getNextWord(roomCode);
8   const secretWord = wordEntry?.word || '';
9   const hint = wordEntry?.impostorClue || '';
10
11  // Inject the word and player info into the payload for the Go
12  // Engine
13  const enrichedPayload = {
14    ...payload, gameId: roomCode, playerIds, requestedImpostors,
15    secretWord, hint
16  };
17  const event = new Event('START_GAME', enrichedPayload);
18  await routingService.handleClientEvent(currentUserId, event);
19  return;
20 }
```

Listing 8: Intercepting START_GAME to fetch the secret word and enrich the payload

4.4.2 In-game text chat and WebRTC signaling relay

Real-time communication is managed by the `ChatAndSignalingService.ts`. For text chat, the gateway routes messages within the `RoomCode`, explicitly excluding the original sender to prevent UI flickering, and also enforces game rules by blocking eliminated players from sending messages. For voice communication, a Peer-to-Peer (P2P) architecture was adopted using WebRTC. The gateway functions strictly as a Signaling Server, relaying cryptographic and network parameters (SDP offers/answers and ICE candidates) without processing audio data.

```
1 if (eventType === 'CHAT') {
2   // Block eliminated players from sending messages
```

```

3      const aliveInRoom = roomAlivePlayers.get(roomCode);
4      if (aliveInRoom && !aliveInRoom.includes(currentUserId)) {
5          ws.send(JSON.stringify({ type: 'error', payload: {
6              message: 'Eliminated_players_cannot_send_messages'
7          }}));
8          return;
9      }
10     const message = new Message(roomCode, currentUserId, payload);
11     await chatAndSignalingService.processChatMessage(message, ws);
12 } else if (eventType === 'WEBRTC') {
13     const message = new Message(roomCode, currentUserId, payload);
14     await chatAndSignalingService.processWebRTCSignaling(message, ws);
15 }

```

Listing 9: Routing Chat and WebRTC signals

4.4.3 Game Engine bridge

The `RoutingService.ts` module implements a Proxy pattern to facilitate communication with the Go-based Game Engine. Client-side WebSocket events are validated and translated into synchronous HTTP POST requests directed to the Engine's REST API.

```

1  // Inside RoutingService.ts
2  async handleClientEvent(userId: string, event: Event): Promise<void> {
3      switch (event.type) {
4          case 'CAST_VOTE':
5              await this.engineGateway.castVote(
6                  String(payload.gameId),
7                  userId,
8                  String(payload.targetId || "")
9              );
10             break;
11          case 'ADVANCE_PHASE':
12              await this.engineGateway.advanceToVoting(String(payload.
13                  gameId));
14              break;
15          // ... handling for other core game events
16      }
17  }

```

Listing 10: Translating WS events to HTTP calls for the Go Engine

Conversely, the gateway exposes an internal HTTP endpoint that listens for incoming Webhooks triggered by the Go Engine. The gateway instantly translates these HTTP payloads back into outward-facing WebSocket events, ensuring that the frontend game state is updated in real time.

4.5 Lobby Service implementation

The Lobby Service serves as the core backend component responsible for the preliminary management of game sessions. It oversees the entire lifecycle of the rooms, handles user authentication profiles, and persists the history of played matches. To guarantee high scalability and maintainability, this module was developed using Java 21 in conjunction with the Spring Boot 4 framework. This technology stack allowed for a rapid environment setup while natively providing enterprise-level patterns. For data persistence, MongoDB was chosen and hosted via MongoDB Atlas. As a document-oriented NoSQL database, it proved to be the ideal solution for dynamically modeling lobby states and game history without the rigid constraints of traditional relational schemas. The interaction with the database is seamlessly handled through Spring Data MongoDB.

4.5.1 Architectural design

From a structural standpoint, the project was organized following the principles of Domain-Driven Design (DDD). This approach effectively isolates the core business logic from infrastructure-specific details, resulting in a codebase divided into three main layers. The domain layer encapsulates the pure entities of the system, such as the `Room` and `User` models, freeing them from any external dependencies. These entities are orchestrated by the application layer through dedicated services, like the `RoomService`, which takes charge of the use cases and manages transactional boundaries. Finally, the infrastructure layer acts as a bridge to the outside world, implementing the REST controllers, physical data access to MongoDB, and various security and configuration filters, including the automated API documentation generated via Swagger and OpenAPI.

By keeping the domain entities rich with behavior, business invariants are inherently protected directly where the data resides. For instance, the `Room` aggregate root independently ensures that no player can join an ongoing game, enforces capacity limits, and guarantees username uniqueness within the lobby itself:

```
1 public void addPlayer(User player) {
2     if (status != RoomStatus.WAITING) {
3         throw new IllegalStateException("Cannot join room: game
4             already started or ended");
5     }
6
7     // Idempotency check
8     if (players.containsKey(player.getId())) {
9         return;
10    }
11
12    if (players.size() >= MAX_PLAYERS) {
13        throw new IllegalStateException("Room is full (max " +
14            MAX_PLAYERS + " players)");
15    }
16
17    // Enforce unique usernames within the same room
18    boolean usernameExists = players.values().stream()
19        .anyMatch(p -> p.getUsername().equalsIgnoreCase(player.
20            getUsername()));
```

```

19     if (usernameExists) {
20         throw new IllegalArgumentException("Username is already
           taken in this room");
21     }
22
23     players.put(player.getId(), player);
24 }

```

Listing 11: Domain entity encapsulating business rules

4.5.2 Room management and security

All the operational dynamics of a lobby are centralized within the dedicated application service. When a player creates a new room, the system initializes its state, generates a unique code, and assigns the creator the role of host. This same service is then responsible for validating every subsequent attempt to join or leave the room, ensuring, for instance, that the game has not already started or that the maximum capacity has not been reached.

The following snippet illustrates how the service handles a player joining a room, combining domain validation, persistence, and inter-service communication:

```

1 public Room joinRoom(String roomCode, User player) {
2     // Retrieve the room or throw an exception if not found
3     Room room = roomRepository.findByCode(RoomCode.of(roomCode))
4         .orElseThrow(() -> new RoomNotFoundException("Room
           with code " + roomCode + " not found"));
5
6     // Domain logic: add player to the room
7     room.addPlayer(player);
8     Room saved = roomRepository.save(room);
9
10    // Prepare data and asynchronously notify the Comm Service
11    List<String> allPlayerIds = saved.getPlayers().stream().map(
        User::getId).toList();
12    commNotifier.notifyPlayerJoined(roomCode, player.getId(),
        player.getUsername(), allPlayerIds);
13
14    return saved;
15 }

```

Listing 12: Room joining logic orchestrating domain and notifications

Regarding security and feature exposure, the service APIs are divided into two categories. On one hand, we have public endpoints queried by the frontend client, which are strictly protected by an authentication system based on JSON Web Tokens (JWT) integrated with Spring Security. On the other hand, internal APIs have been designed purely for direct communication between the various containers of the system. These restricted endpoints allow other services, such as the Game Engine or the Comm Service, to bypass certain public restrictions to perform rapid validations or state updates safely within the local Docker network.

4.5.3 Integration with the Comm Service

One of the main challenges in developing this component was maintaining constant synchronization between the backend state and the state perceived by clients connected in real time. Whenever a lobby's state is altered via a REST call, it is crucial that this information is propagated immediately.

To address this issue, the Lobby Service implements a push notification mechanism directed at the Comm Service. This system leverages Spring's `RestClient` to send webhook requests following a "fire-and-forget" pattern. In practice, whenever a relevant event occurs, the system triggers an asynchronous notification in a separate thread.

```
1 @Async
2 public void notifyPlayerJoined(String roomCode, String playerId,
3     String username, List<String> allPlayerIds) {
4     sendNotification(CommEvent.PLAYER_JOINED,
5         new PlayerJoinedPayload(CommEvent.PLAYER_JOINED,
6             roomCode, playerId, username, allPlayerIds));
7 }
8
9 private void sendNotification(CommEvent event, Object payload) {
10     try {
11         restClient.post()
12             .uri(event.getPath())
13             .contentType(MediaType.APPLICATION_JSON)
14             .body(payload)
15             .retrieve()
16             .toBodilessEntity();
17     } catch (Exception e) {
18         // Fire-and-forget: we do not block the Lobby Service if
19         // Comm Service is unreachable
20         log.warn("Failed to notify comm-service: {}", e.
21             getMessage());
22     }
23 }
```

Listing 13: Asynchronous, fire-and-forget notification implementation

Communications are carried out by sending strongly typed payloads, utilizing the *Record* structures introduced in recent Java versions, thereby avoiding formatting errors in the transmitted JSON data. Particular attention was also paid to the resilience of this integration. Strict connection and read timeouts have been configured so that any slowdown or outage in the Comm Service does not cause a cascading failure in the Lobby Service operations. If a notification fails, the exception is quietly handled in the server logs, allowing the main operation to conclude successfully and the user to receive a timely response.

5 DevOps

To ensure a smooth collaboration among team members, maintain high code quality, and automate repetitive tasks across our diverse microservices architecture, we adopted a modern DevOps and Continuous Integration/Continuous Delivery (CI/CD) strategy.

5.1 DVCS workflow

For version control, we adopted GitHub Flow, a lightweight, branch-based workflow centered around a protected `main` branch. The `main` branch always represents the stable, deployable state of our game.

To prevent regressions, we enabled **Branch Protection Rules**: direct pushes to `main` are strictly forbidden. Development of new features or bug fixes occurs in dedicated feature branches. Once a feature is complete, a Pull Request (PR) is opened. The code can only be merged or rebased into `main` after passing all automated CI checks and receiving necessary approvals.

5.2 Release automation

5.2.1 Conventional commits

To maintain a clear and predictable project history, we strictly adhere to a rigorous implementation of the Conventional Commits standard. Every commit message must follow a specific, mandatory structure: a standard type (e.g., `feat`, `fix`, `chore`), followed by an obligatory scope enclosed in parentheses (e.g., `(docker)`, `(deps)`, `(core)`), a colon, and a concise description of the change.

An acceptable commit message must look exactly like this: *“chore(deps): update dependency node to v22”*.

5.2.2 Semantic release

Manual versioning is prone to human error. To solve this, we integrated **semantic-release** into our pipeline. By analyzing the Conventional Commits pushed to the `main` branch, semantic-release automatically determines the next semantic version number (Major, Minor, or Patch), generates an accurate `CHANGELOG.md`, and publishes the official GitHub Release without any human intervention.

5.3 Quality assurance

5.3.1 Husky and Commitlint

To enforce our strict commit standards locally before code even reaches GitHub, we configured Git hooks using **Husky**. Specifically, we utilize the `commit-msg` hook integrated with **commitlint**. Whenever a developer attempts to make a commit, Husky intercepts it and executes `npm run commitlint -- -edit "$1"`. This command validates the message against our custom Conventional Commits rules. If the format is invalid—for instance, if a developer forgets the mandatory scope parentheses or the colon (e.g., writing `fix bug` instead of `fix(ui): resolve button alignment`)—the commit is immediately aborted.

5.3.2 Linting and formatting

Code quality is further ensured by automated linters (such as **Oxlint** and **ESLint** for our Node/Vue services). These tools are executed automatically to catch syntax errors and enforce consistent coding styles across the entire monorepo, allowing the team to focus on logic rather than formatting debates.

5.4 GitHub Actions workflows: CI and CD

Given the polyglot nature of our monorepo (Java, Go, Node.js, and Vue.js), we implemented a robust CI/CD pipeline using GitHub Actions. To maintain a clean architecture, we established a strict separation of concerns by splitting our automation into two distinct workflows:

5.4.1 Continuous Integration (CI) pipeline

Whenever a Pull Request is opened or updated, the `ci.yml` workflow triggers a series of parallel automated checks to ensure code quality and system stability.

- **Commit Message validation:** the pipeline enforces the *Conventional Commits* specification by running a linting job on all commit messages within the Pull Request. This guarantees a clean, standardized Git history, which is a strict prerequisite for the automated *Semantic Release* process.
- **Multi-language Build verification:** the workflow independently compiles and tests the different stacks within the monorepo. It specifically targets the Lobby Service (Java 21 via Gradle), the Game Engine (Go 1.26), and the Comm-Service/Frontend (Node.js 20).
- **Containerization checks:** for the backend microservices, the pipeline verifies that their respective `Dockerfiles` build successfully. This “dry-run” ensures that no code integration breaks the containerization process or the target deployment environment.

Only when all these automated checks return a green status is the Pull Request allowed to be merged into the `main` branch.

5.4.2 Continuous delivery and versioning (`release.yml`)

This workflow runs *exclusively* when code is successfully merged into the `main` branch. It is responsible for the final stage of our pipeline: it executes **semantic-release**, which calculates the new version number, automatically generates the `CHANGELOG.md`, and creates the official GitHub Release. By separating these two workflows, we ensure that testing happens continuously in isolated branches, while release generation is safely restricted to the stable `main` branch.

5.5 Dependency management

To prevent software decay and mitigate security vulnerabilities, we integrated **Renovate Bot**. Renovate continuously monitors our dependency files (`package.json`, `build.gradle`,

`go.mod`, and GitHub Actions) and automatically opens Pull Requests when minor updates or patches are available. We configured the bot to respect our architectural choices (e.g., pinning to LTS versions like Java 21 and Node 22), avoiding disruptive major updates while keeping the underlying infrastructure secure and up to date.

5.6 Local development environment

To orchestrate the complex interaction between our microservices during local development, we utilize **Docker Compose**. By executing a single command (`docker compose up`), developers can instantly spin up the entire game infrastructure—including the Java Lobby, Go Engine, Node WebSocket server, and Vue frontend—connected through a dedicated virtual network. This eliminates the ‘it works on my machine’ problem and drastically reduces the setup time for new contributors.

6 License

The decision to release the source code as *open-source* software was primarily driven by the desire to benefit from the free *Continuous Integration* and *Continuous Deployment* tools (specifically GitHub Actions) available for public repositories. In this context, the project’s objective is to allow third parties to study, modify, redistribute, and integrate the software into other ecosystems, including proprietary ones. For these reasons, we opted for the **MIT License**, an extremely permissive license that grants maximum freedom of reuse, provided that the original copyright notice is retained. Although the developed application logic and its underlying algorithms are governed by the MIT License, the system is designed to be modular and data-agnostic. Should the software integrate or process external assets or *datasets* subject to different licenses (such as *Creative Commons* licenses), the software rights will remain distinct from the data usage rights, ensuring strict compliance with the attribution clauses required by the respective providers.

7 Deployment

The application deployment adopts a container-based approach, ensuring isolation, reproducibility, and independence from the host environment. The entire architecture is defined declaratively via a `docker-compose.yml` file, which orchestrates service execution within a dedicated virtual network.

7.1 Container architecture

The configuration includes the following microservices:

- **lobby-service**: the Java Spring Boot backend. It handles user authentication, session management, and room creation. Configuration occurs dynamically at runtime via environment variables (such as the `MONGODB_URI`).
- **game-engine**: the Go high-performance state manager. It is responsible for executing the core game logic, handling round timers, and validating player turns.
- **comm-service**: the Node.js service that acts as a real-time hub. It manages persistent WebSocket connections and coordinates WebRTC signaling.

- **front-end:** the Vue.js Single Page Application. Within this local deployment configuration, it is served via the Vite development server. This allows for rapid testing while keeping the runtime environment fully containerized and isolated from the host machine.

7.2 Multi-stage builds

For the compiled backend services (Java and Go), a multi-stage build strategy defined in their respective `Dockerfiles` was adopted:

- **Build:** A heavy base image (e.g., Go compiler, JDK) compiles the source code and resolves dependencies.
- **Serve:** The compiled artifacts are copied into a lightweight runtime image (e.g., Alpine Linux, JRE). All source code and development dependencies are discarded, drastically reducing the final container size and minimizing the security attack surface.

7.3 Setup and execution

System deployment requires Docker and Docker Compose installed on the host machine.

1. **Clone the repository:** download the project source code by cloning the GitHub repository:

```
git clone https://github.com/MarcoCostantini26/TheImpostor.git
```

2. **Configuration:** create a `.env` file in the project root containing the necessary environment variables, such as the MongoDB URI and internal service URLs.
3. **Startup:** the following command builds the images and starts the entire stack in the background:

```
docker compose up -d --build
```

4. **Access:** once initialization is complete, the entire platform is accessible to the end user at `http://localhost:5173`, while the backend services remain strictly isolated within the internal `game-network` Docker bridge.